L8-v6

2026年10月22日 16:14

代码地图（Repo Map）

一、概念

定义：代码地图是一份用极少的 token 描述整个仓库结构的文本摘要。它告诉模型：这个仓库里有哪些文件、每个文件里定义了哪些符号（类/函数/结构体）、谁依赖谁、哪些是核心。

它解决的问题：

场景	没有代码地图	有代码地图
模型刚接手一个 500 文件的仓库	不知道从哪下手，只能盲目 `ls` + `cat`	一眼看到骨架，直接定位相关文件
用户问"`process_data` 在哪定义"	模型要 grep 整个仓库，烧掉几万 token	地图里直接有 `function process_data @ 某文件`
改一个函数前要判断影响面	模型不知道谁调用了它	地图里能看到依赖方向

它在上下文分层中的位置：`L2_RepoMap` 层，排在 `L0_System` 和 `L1_Project` 之后、`L3_Retrieved` 和 `L4_ActiveFiles` 之前。这个位置的含义是"会话内基本不变、但比系统提示更具体"——它属于缓存前缀的一部分，改一次会让缓存失效，所以只在构建时和活动文件变化时刷新。

二、基本原理

2.1 核心矛盾

模型的上下文窗口有限（比如 200K token），而仓库规模往往远超这个限制（500 个文件 × 平均 2000 token = 100 万 token）。不可能把整个仓库塞进去。

两个朴素方案都不行：

方案 A：让模型自己探索。 模型 `ls` → `cat a.cpp` → `cat b.cpp` → ... 每个文件都要一次工具调用 + 一次模型推理，500 个文件要几百轮，烧掉几十万 token 且极慢。

方案 B：向量检索。 把每个函数做成 embedding，按用户问题检索 top-K。这个方案在代码上有一个根本缺陷——见下一节。

2.2 为什么代码需要"结构"而不是"语义相似度"

缺陷一：代码的相关性是结构性的，不是语义的。

```
void process_data() {
    auto user = load_user(id);   // process_data 的正确性依赖 load_user
    // ...
}
```

`process_data` 和 `load_user` 在自然语言语义上可能毫无共同关键词——一个处理数据、一个加载用户。但 `process_data` 的正确性依赖于 `load_user` 的契约（返回值格式、异常行为）。向量检索抓不到"谁调用谁"。

缺陷二：相似度会把不同的东西混在一起。

```
void foo(int x);
void foo(std::string s);
```

两个重载的 embedding 几乎重合（都是"一个叫 foo 的函数"），但它们是两个不同的函数，签名不同、行为不同、调用点不同。向量检索会把它们当成一回事。

结论：代码需要的是依赖图（谁 include 谁、谁引用谁的符号），不是语义相似度。

2.3 三要素

一张有用的代码地图必须回答三个问题：

问题	对应技术	输出
仓库里有什么？	符号抽取	每个文件的类/函数/结构体清单
谁依赖谁？	依赖建图	文件之间的有向边
哪些最重要？	重要性排序	每个文件的分数

三者缺一不可：

- 只有符号清单 → 一张平铺的列表，模型不知道从哪看起。
- 只有依赖图 → 模型看不到具体内容。
- 没有重要性排序 → 预算不够时不知道截掉谁。

2.4 为什么用 PageRank 而不是简单计数

最直觉的"重要性"是入度（被多少个文件 include）。它的缺陷是只看数量，不看质量：

A ← B　（B 是个没人用的工具文件）
A ← C　（C 是核心文件，被 50 个文件依赖）

简单计数：A 的入度 = 2，看起来一般。

PageRank：从 C 传过来的权重远大于从 B 传过来的——因为 C 本身很重要。

PageRank 的直觉：一个文件的重要性 = 所有依赖它的文件的重要性之和（加权）。重要的文件把重要性"传递"给它依赖的文件。

数学形式（个性化 PageRank）：

```
rank[t+1][j] = d · Σ_i (rank[t][i] · w(i→j) / outDeg(i))
             + (d · dangling + (1-d)) · p[j]
```

- d = 0.85：阻尼系数，模拟"用户有 15% 概率随机跳到任意页面"。
- p[j]：传送向量，默认均匀分布（1/n），可被"活动文件偏置"修改。
- dangling：所有出度为 0 的节点的 rank 之和（它们没有出边，质量要均分回去）。
- 归一化约束：Σ rank[j] = 1。

"个性化"的含义：p[j] 可以被偏置。如果用户正在编辑 foo.cpp，就把 p[foo.cpp] 设大——这样 foo.cpp 及其依赖链上的文件 rank 会升高，地图里优先显示它们。这就是"活动文件偏置"。

三、主要技术

3.1 符号抽取

技术	原理	优点	缺点
正则/词法	逐行扫，找"看起来像声明"的行	零依赖、快	漏模板、命名空间内方法、构造函数、operator 重载
tree-sitter	用真正的语法分析器解析 AST	准确，能抽全	需要编译文法、有依赖
LSP / clangd	调语言服务器	最准确（含类型信息）	重、慢、需要完整编译环境

3.2 依赖建图

技术	原理	优点	缺点
include/import 解析	抓 `#include "..."` / `import ...`	精确，是编译期硬依赖	只覆盖显式依赖
符号名匹配文件名	文件 A 定义了 `class Alpha`，文件 `alpha.cpp` 存在 → A → alpha.cpp	覆盖"命名约定互相索引"	启发式，会误判
调用图	解析函数体，找谁调用了谁	最精确的运行时依赖	需要完整类型推断，极重
全仓库 grep	搜符号名出现在哪些文件	简单	慢、有噪声、无方向

3.3 重要性排序

技术	原理	优点	缺点
入度计数	数被多少个文件依赖	简单	不看依赖者的质量
PageRank	迭代传递重要性	考虑依赖者的质量	需要迭代，有数值陷阱
HITS	同时算 hub 和 authority	区分"被依赖"和"依赖别人"	同样要迭代，更复杂
TF-IDF	符号名在仓库里的稀有度	抓"独特符号"	与依赖结构无关

3.4 渲染

技术	原理	优点	缺点
按 rank 排序 + 截断	核心符号优先，超预算就停	简单、可预期	可能截掉一个文件的后半部分

按文件分组 + 每组限额	每个文件最多显示 N 个符号	保证每个文件都有代表	可能挤掉核心文件的次要符号
分层渲染	预算充足时给完整签名，紧张时只给符号名	预算利用率高	实现复杂

四、L8采用的技术

环节	选择	理由
符号抽取	tree-sitter + 词法兜底	真解析器能抽全；词法兜底保证没编 tree-sitter 时也能跑，且提供 A/B 对比
依赖建图	include 边（1.0）+ 符号名匹配边（0.5）	覆盖两种主要依赖；权重区分可信度
重要性排序	个性化 PageRank（d=0.85，20 轮，活动文件偏置×8）	考虑依赖质量；支持"正在处理的文件优先"
渲染	按 rank 排序 + 按字符预算截断（tokenBudget × 4）	简单可预期；4 字符 ≈ 1 token 是经验值

五、tree-sitter ：“编辑器”解析器

传统解析器（bison、ANTLR）面向批处理：读入一个完整的文件，从头解析一遍，产出 AST，然后结束。

tree-sitter 面向编辑器：文件永远处于“半输入”状态，每次按键都要重新解析，而且即使括号不配对、表达式不完整，也必须产出一棵可用的树。

它的四个设计目标直接决定了它的 API 形状：

目标	机制	对使用者的含义
通用	GLR（广义 LR）解析	几乎任何语言都能写文法，冲突在运行时探索，不在生成时报错
快速	增量解析，复用未改动的子树	改一个字符，只重新解析受影响的部分
鲁棒	错误恢复，产出 ERROR / MISSING 节点	文件语法错误时不抛异常，而是返回一棵带错误标记的树
零依赖	纯 C11 运行时	可以嵌入任何宿主程序，包括你的 C++ Agent

输出是一棵具体语法树（CST），而不是抽象语法树（AST）。CST 保留了每一个 token 在原文中的字节位置和长度，代价是节点更多、树更大，但换来的是“从节点能精确还原源码片段”的能力。

renderMap 输出的是一份"按文件分组、按 PageRank 排序、按字符预算截断"的符号清单。每个符号只有两个字段：kind（class/struct/enum/function/method）和 signature（函数是"返回类型 全限定名(参数)"，类是"全限定名 [基类]"，枚举是"全限定名"）。不输出行号、不输出依赖关系、不输出函数体——地图负责"有什么"，Read 工具负责"在哪、长什么样"。